CN Project

Ujwal Sanikam L PES1UG24CS506
Ved Mudkavi PES1UG24CS522

Phase 1: The Basic Connection (Proof of Concept)

- Creating a basic Server that binds to a port and listens.

- Creating a Client that connects to the server.

- Have the server send a simple text message ("Hello Client") and ensure the client receives it.

```
D:\CN PROJECT>python server.py
Server is listening on 127.0.0.1:65432...
Success! Connected by client at ('127.0.0.1', 50734)
Message sent to client. Shutting down server.
```

```
D:\CN PROJECT>python client.py
Attempting to connect to server at 127.0.0.1:65432...
Received from server: Hello Client! The TCP connection is successful.
```

server.py > ...

```python
import socket

# 1. Setup server details
HOST = '127.0.0.1' # Localhost (meaning it runs on your own machine)
PORT = 65432       # Port to listen on (we use a high number to avoid system ports)

# 2. Create a TCP/IP socket
# AF_INET means IPv4, SOCK_STREAM means TCP
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server_socket:

    # 3. Bind the socket to the address and port
    server_socket.bind((HOST, PORT))

    # 4. Start listening for incoming connections
    server_socket.listen()
    print(f"Server is listening on {HOST}:{PORT}...")

    # 5. Accept a connection (The script will pause/block here until a client connects)
    conn, addr = server_socket.accept()

    with conn:
        print(f"Success! Connected by client at {addr}")

        # 6. Send a test message
        message = "Hello Client! The TCP connection is successful."

        # Sockets can only send bytes, not strings, so we must encode it first
        conn.sendall(message.encode('utf-8'))
        print("Message sent to client. Shutting down server.")
```

client.py > ...

```python
import socket

# 1. Setup connection details (These MUST match the server)
HOST = '127.0.0.1'
PORT = 65432

# 2. Create a TCP/IP socket
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as client_socket:
    print(f"Attempting to connect to server at {HOST}:{PORT}...")

    # 3. Connect to the server
    client_socket.connect((HOST, PORT))

    # 4. Receive data from the server (Wait to receive up to 1024 bytes)
    data = client_socket.recv(1024)

# 5. Decode the bytes back into a readable string and print it
print(f"Received from server: {data.decode('utf-8')}")
```

TCP handshake has been created and now the TCP port is working

Phase 2: Single Client Streaming & Buffering

- Upgrade the server to open a .wav or .mp3 file.
Implement a loop that reads a small chunk (e.g., 4096 bytes) and sends it over the TCP socket.

- Have the client receive those bytes and save them to a local file, or pipe them directly to an audio playback library (like PyAudio in Python).

- If we have a 50MB audio file, we cannot load the entire thing into the server's RAM and blast it over the network all at once. It would overwhelm the client's network connection and eat up memory.

- Instead, we are using a buffer. The server reads a tiny "chunk" of the audio file (e.g., 4096 bytes), sends it, reads the next chunk, sends it, and repeats until the file is done.

```
D:\CN PROJECT>python server.py
Server listening on 127.0.0.1:65432...
Connected by ('127.0.0.1', 54139)
Starting to stream sample.mp3...
Finished streaming audio file.
```

```
D:\CN PROJECT>python client.py
Connecting to 127.0.0.1:65432...
Receiving audio stream...
Stream complete. Audio saved as 'received_audio.mp3'.
```

server.py > ...

```python
import socket
import os

HOST = '127.0.0.1'
PORT = 65432
FILENAME = 'sample.mp3'  # Make sure this file is in your folder!
CHUNK_SIZE = 4096        # Read 4KB at a time

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server_socket:
    server_socket.bind((HOST, PORT))
    server_socket.listen()
    print(f"Server listening on {HOST}:{PORT}...")

    conn, addr = server_socket.accept()
    with conn:
        print(f"Connected by {addr}")

        # Check if the file exists before trying to send it
        if not os.path.exists(FILENAME):
            print(f"Error: {FILENAME} not found.")
        else:
            print(f"Starting to stream {FILENAME}...")

            # Open the audio file in Binary Read mode ('rb')
            with open(FILENAME, 'rb') as audio_file:
                while True:
                    # Read a chunk of the file
                    audio_chunk = audio_file.read(CHUNK_SIZE)

                    # If the chunk is empty, we reached the end of the file
                    if not audio_chunk:
                        break

                    # Send the binary chunk to the client
                    conn.sendall(audio_chunk)

            print("Finished streaming audio file.")
```

client.py > ...

```python
import socket

HOST = '127.0.0.1'
PORT = 65432
OUTPUT_FILENAME = 'received_audio.mp3'
CHUNK_SIZE = 4096

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as client_socket:
    print(f"Connecting to {HOST}:{PORT}...")
    client_socket.connect((HOST, PORT))

    print("Receiving audio stream...")

    # Open a new file in Binary Write mode ('wb') to save the incoming data
    with open(OUTPUT_FILENAME, 'wb') as audio_file:
        while True:
            # Receive up to CHUNK_SIZE bytes from the server
            data = client_socket.recv(CHUNK_SIZE)

            # If the server stops sending data, the stream is over
            if not data:
                break

            # Write the received bytes to the local file
            audio_file.write(data)

    print(f"Stream complete. Audio saved as '{OUTPUT_FILENAME}'.")
```

Phase 3: Multi-threading (Handling Multiple Listeners)

- Currently, our server is a "blocking" server. The server waits for a client. The client connects. The server starts sending the audio file chunk by chunk.

- The Problem: While it is sending that file, it cannot do anything else. If a second client tries to connect, the server will completely ignore them until the first client finishes downloading the entire song.

- Multi-threading:
Instead of the main server doing the heavy lifting, we turn it into a main thread. The main thread waits for a client. When a client 'A' connects, the main thread spawns a new thread (thread 1). The main thread hands over the client 'A' to thread 1. Thread 1 takes care of streaming the audio to client 'A'. The main thread immediately goes back and waits for its next client.

When the server creates new threads

```
Microsoft Windows [Version 10.0.26200.8039]
(c) Microsoft Corporation. All rights reserved.

D:\CN PROJECT>python server.py
[LISTENING] Server is listening on 127.0.0.1:65432...

[NEW CONNECTION] Client ('127.0.0.1', 61913) connected.
[ACTIVE STREAMS] 1

[FINISHED] Done streaming to ('127.0.0.1', 61913).

[NEW CONNECTION] Client ('127.0.0.1', 61914) connected.
[ACTIVE STREAMS] 1

[FINISHED] Done streaming to ('127.0.0.1', 61914).
```

When two separate clients are run to connect to the server at the same time

```
Microsoft Windows [Version 10.0.26200.8039]
(c) Microsoft Corporation. All rights reserved.

D:\CN PROJECT>python client.py
Connecting to 127.0.0.1:65432...
Receiving audio stream...
Stream complete. Audio saved as 'received_audio_2593.mp3'.

D:\CN PROJECT>python client.py
Connecting to 127.0.0.1:65432...
Receiving audio stream...
Stream complete. Audio saved as 'received_audio_6080.mp3'.

D:\CN PROJECT>
```

```python
import threading  # NEW: We need this to handle multiple clients at once!

HOST = '127.0.0.1'
PORT = 65432
FILENAME = 'sample.mp3'  # Make sure this matches your working audio file!
CHUNK_SIZE = 4096

# NEW: We created a "Worker" function. Each new client gets their own worker.
def handle_client(conn, addr):
    print(f"\n[NEW CONNECTION] Client {addr} connected.")
    try:
        with open(FILENAME, 'rb') as audio_file:
            while True:
                audio_chunk = audio_file.read(CHUNK_SIZE)
                if not audio_chunk:
                    break
                conn.sendall(audio_chunk)
        print(f"\n[FINISHED] Done streaming to {addr}.")
    except Exception as e:
        print(f"\n[ERROR] Connection with {addr} lost: {e}")
    finally:
        conn.close()

# MAIN SERVER LOGIC (The "Manager")
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server_socket:
    server_socket.bind((HOST, PORT))
    server_socket.listen()
    print(f"[LISTENING] Server is listening on {HOST}:{PORT}...")

    # NEW: An infinite loop. The server will stay on forever, accepting new clients.
    while True:
        conn, addr = server_socket.accept()

        # When a client connects, create a new Thread (Worker) for them
        thread = threading.Thread(target=handle_client, args=(conn, addr))
        thread.start()

        # Print how many clients are currently streaming (subtract 1 for the main Manager thread)
        print(f"[ACTIVE STREAMS] {threading.active_count() - 1}")
```

D: > CN PROJECT > client.py > ...

```python
import socket
import random  # NEW: To generate a unique file name

HOST = '127.0.0.1'
PORT = 65432
# NEW: Give this client a random ID (e.g., received_audio_4921.mp3)
client_id = random.randint(1000, 9999)
OUTPUT_FILENAME = f'received_audio_{client_id}.mp3'
CHUNK_SIZE = 4096

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as client_socket:
    print(f"Connecting to {HOST}:{PORT}...")
    client_socket.connect((HOST, PORT))

    print("Receiving audio stream...")

    with open(OUTPUT_FILENAME, 'wb') as audio_file:
        while True:
            data = client_socket.recv(CHUNK_SIZE)
            if not data:
                break
            audio_file.write(data)

    print(f"Stream complete. Audio saved as '{OUTPUT_FILENAME}'.")
```

Phase 4: Adaptive Streaming Logic & QoS (Quality of Service) Evaluation.

- Right now, our server blindly blasts audio chunks as fast as possible. But in the real world, Wi-Fi fluctuates. If the network gets slow, the client's audio will stop and buffer (which users hate).

To fix this, we have done 2 things:

- Quality of Service(QOS) Evaluation (Client Side): The client acts like a speed camera. It times how long it takes to receive a batch of chunks. If it takes too long, it realizes the network is struggling.

- Adaptive Streaming (Server Side): The TCP connection is a two-way street. The client will send a message back to the server saying "LOW_Q" (Low Quality). The server will hear this and "adapt" to save the stream.

```
C:\Windows\System32\cmd.e

D:\CN PROJECT>python client.py
Connecting to 127.0.0.1:65432...
Receiving audio stream and monitoring QoS...

Received chunk 1...
Received chunk 2...
Received chunk 3...
Received chunk 4...
Received chunk 5...
Received chunk 6...
Received chunk 7...
Received chunk 8...
Received chunk 9...
Received chunk 10...

  >>> [Good] 10 chunks took 0.10s. Requesting HIGH quality.

Received chunk 11...
Received chunk 12...
Received chunk 13...

Stream complete. Audio saved.

D:\CN PROJECT>python client.py
Connecting to 127.0.0.1:65432...
Receiving audio stream and monitoring QoS...

Received chunk 1...
Received chunk 2...
Received chunk 3...
Received chunk 4...
Received chunk 5...
Received chunk 6...
Received chunk 7...
Received chunk 8...
Received chunk 9...
Received chunk 10...

  >>> [Warning] 10 chunks took 4.51s. Requesting LOW quality.

Received chunk 11...
Received chunk 12...
Received chunk 13...
```

- Buffer Management: Implemented chunk-by-chunk reading and writing.

- QoS Evaluation: Built a client-side stopwatch to measure network latency.

- Adaptive Streaming Logic: Created a two-way communication channel where the server dynamically reacts to the client' network health.

```
C:\Windows\System32\cmd.e

Microsoft Windows [Version 10.0.26200.8039]
(c) Microsoft Corporation. All rights reserved.

D:\CN PROJECT>python server.py
[LISTENING] Server is listening on 127.0.0.1:65432...

[NEW CONNECTION] Client ('127.0.0.1', 62970) connected.
[('127.0.0.1', 62970)] QoS ALERT: Network slow. Adapting to LOW bitrate...

[FINISHED] Done streaming to ('127.0.0.1', 62970).
```

Now the server also prints the QOS alert and adapts to the low network